HashMap Interview Handbook

WHY ➜ HOW ➜ INTERNALS ➜ INTERVIEW. Written in the same style we learned together.

1. WHY WAS HASHMAP CREATED?

Imagine 10 lakh employees stored in an ArrayList. Finding employee id 874512 requires checking one by one (O(n)). HashMap was created to avoid searching every element. Its goal is simple: jump close to the data instead of scanning everything.

2. INTERNAL STRUCTURE

Internally HashMap uses an array called table[]. Every position in this array is called a bucket.

table[] -> [0][1][2][3][4][5]...
Bucket 5 may contain one node or multiple nodes.

3. BUCKET = ARRAY INDEX

One of the biggest confusions: bucket number is simply the array index. HashMap never creates an array of size equal to the hash code. The hash code is converted into a valid array index using an internal calculation.

4. PUT() FLOW

Key -> hashCode() -> Bucket Index -> Is bucket empty? If yes, create a new node. If not, compare keys using equals(). If the same key is found, overwrite the value; otherwise create another node in that bucket.

5. GET() FLOW

Key -> hashCode() -> Bucket Index -> Traverse nodes in that bucket -> equals() finds the exact key -> return value.

6. HASH COLLISION

Collision means two different keys reach the same bucket. HashMap stores both nodes in the same bucket. Java 8+ uses a linked list first and may later convert it into a Red-Black Tree.

7. hashCode() vs equals()

Remember forever: hashCode() decides WHICH BUCKET. equals() decides WHICH KEY inside that bucket. hashCode() alone is not enough because two different keys may have the same hash code.

8. CUSTOM OBJECTS

If Student(1,'Rahul') and another Student(1,'Rahul') should represent the same logical key, override BOTH equals() and hashCode(). If you override only equals(), equal objects may go into different buckets and never be compared.

9. CAPACITY, LOAD FACTOR & THRESHOLD

Default capacity = 16. Default load factor = 0.75. Threshold = capacity × load factor = 12. When the 13th element is inserted, HashMap resizes.

10. RESIZE & REHASH

Resize does not mean simply creating a larger array. Java creates a new array (usually double the capacity) and recalculates the bucket index for every existing node. Some nodes remain in the same bucket while others move to new buckets.

11. TREEIFICATION (Java 8+)

LinkedList changes to Red-Black Tree ONLY when BOTH conditions are true: bucket node count >= 8 AND table capacity >= 64. Otherwise HashMap prefers resizing.

12. HASHSET CONNECTION

HashSet internally uses HashMap. Your value becomes the HashMap key and a dummy PRESENT object becomes the value. Therefore HashSet gets hashing, collision handling and resizing for free.

13. TIME COMPLEXITY

Average put/get/remove = O(1). Worst case = O(n) when many collisions happen. With treeification, worst bucket operations improve to O(log n).

14. COMMON MISTAKES

• Bucket is not a separate object; think of it as an array index. • hashCode() does not identify the exact key. • Forgetting to override both equals() and hashCode() for custom keys. • Thinking every resize keeps nodes in the same bucket.

15. REMEMBER FOREVER

HashMap = Array + Buckets + Nodes. Bucket = Array Index. hashCode() -> Bucket. equals() -> Exact Key. Collision -> Same bucket. Threshold = Capacity × Load Factor. Resize -> Rehash all nodes. Treeification: bucket>=8 AND capacity>=64.